

Évaluation Python & FastAPI

2 h 30 · Travail individuel · Note sur 20

Créez une petite API de gestion de stations de vélos, comme dans les TP Dock Control. Partez d'un nouveau dossier **eval-nom-prenom-python-b2/**. Aucun starter n'est nécessaire.

Utilisez Python 3.12, FastAPI, Pydantic, SQLAlchemy et pytest. SQLite suffit. Aucun frontend ni système de connexion à développer.

Temps conseillé : exercices 1 à 4 : 2 h 15 · vérification et rendu : 15 min.

Exercice 1 · Comprendre le contrat HTTP

15 min · 3 points (1 point par réponse)

Répondez dans **reponses.md**, en une ou deux phrases par question.

1. Une route **GET /stations** crée une station. Quel verbe et quel code HTTP faut-il utiliser pour cette création ?
 2. **GET /stations/999** demande une station inexistante. Quel code HTTP et quel type de réponse faut-il renvoyer ?
 3. Quelle est la différence entre les codes **401** et **403** ? Donnez un exemple de chaque cas.
-

Exercice 2 · Construire l'API

50 min · 7 points

Une station : id entier généré ; code non vide et unique ; name non vide ; capacity entier ≥ 1 ; status parmi open, closed, maintenance (open par défaut).

1. Créez **app/main.py** et **requirements.txt**. Ajoutez **GET /health** : 200 et {"status": "ok"}. **(1 pt)**
2. Créez un schéma Pydantic pour les données d'une station, sans id en entrée. Une donnée invalide doit produire **422**. **(2 pts)**
3. Ajoutez **POST /stations** : 201 et la station créée. Un code déjà utilisé doit produire **409**. **(1,5 pt)**
4. Ajoutez **GET /stations** avec le filtre optionnel **?status=open**, puis **GET /stations/{id}** : 200 si trouvée, 404 sinon. **(1,5 pt)**
5. Ajoutez **PATCH /stations/{id}** pour modifier uniquement name ou status. Les champs absents restent inchangés ; id inconnu : 404 ; données invalides : 422. **(1 pt)**

Pour cet exercice, une liste en mémoire suffit. Toutes les réponses de station contiennent id, code, name, capacity et status. Les erreurs sont en JSON ; le format standard de FastAPI est accepté.

Exercice 3 · Enregistrer les stations

30 min · 4 points

1. Créez une table stations avec **SQLAlchemy** et **SQLite** : id clé primaire, code unique. **(1 pt)**
2. Branchez les routes de l'exercice 2 sur la base. Fermez les sessions après utilisation. **(1,5 pt)**
3. En cas d'**IntegrityError** sur un doublon, faites un **rollback** puis renvoyez **409**. **(1 pt)**
4. Vérifiez qu'une station existe encore après redémarrage. Notez le résultat dans **reponses.md**. **(0,5 pt)**

Exercice 4 · Tester l'API

40 min · 6 points

1. Créez une fixture **client** avec **TestClient** et une base SQLite de test distincte de la base de l'application. Chaque test démarre avec une base vide (en mémoire ou dans un fichier temporaire). **(1,5 pt)**
2. Écrivez les tests ci-dessous avec **pytest** : vérifiez les codes HTTP et les données utiles du JSON. **(4 pts)**
3. Lancez **python -m pytest -q** deux fois de suite. Notez les deux réussites dans **reponses.md**. **(0,5 pt)**

Cas à tester

- GET /health : 200 et status égal à ok.
- Création puis lecture par id : 201, puis 200 avec les mêmes données.
- Filtre status=open : seules les stations open sont renvoyées.
- PATCH du name : le nom change ; code, capacity et status restent identiques.
- Lecture d'un id inexistant : 404 et réponse JSON.
- Entrées invalides : capacity=0 et status="flying" produisent chacune 422, sans créer de station.
- Deux créations avec le même code : la seconde produit 409 ; une seule station existe.

Les tests appellent l'API et sont indépendants de leur ordre. Un test paramétré convient pour les entrées invalides.

Vérification et rendu

15 min · inclus dans les 2 h 30

1. Ajoutez un **README.md** avec les commandes d'installation, de lancement de l'API et des tests.
2. Enregistrez votre projet dans un dépôt **GitHub** nommé **eval-nom-prenom-python-b2** : app/, tests/, requirements.txt, README.md, reponses.md et configuration nécessaire.
3. Excluez .env/, les caches Python et les bases de données. La base doit être recrée au lancement.
4. Envoyez le **lien du dépôt au formateur sur Discord**. Assurez-vous qu'il peut y accéder. Rendez votre travail même s'il est incomplet.

Grille de notation

Points attribués selon les critères indiqués dans chaque exercice.

1 · Contrat HTTP	Verbes et codes HTTP	3
2 · API	Health, validation, création, lectures et modification	7
3 · Persistance	Table, accès en base, rollback et redémarrage	4
4 · Tests	Fixture : 1,5 · cas de test : 4 · deux exécutions : 0,5	6
Total		20 / 20

Tests (4 pts) : 0,5 par cas, sauf les deux entrées invalides : 1 pt au total.